The Name Is the Contract: A Promotion Gate Called `mcnemar_exact` Is Not Exact — and, Against the Two-Sided Exact Reference, Every Flip Favors Promotion

Zewen Liu
Independent Researcher
17353895263@163.com

Abstract

A self-improving agent system accepts a candidate variant through an acceptance rule, and the rule's name is the interface every downstream consumer reads. We audit one such rule: a function named `mcnemar_exact` that gates self-evolution promotion via `promote = bool(new_wins > old_wins and p < 0.05)`. Its docstring admits the body's reality (exact below n=25, normal approximation otherwise), but the name advertises a rigour the body does not honour. Recomputing the entire fallback region (n=26 . . . 500, $b \geq c$: 62,819 cells, of which the gate would promote 55,469) against the level-matched exact reference (`scipy.stats.binomtest`, two-sided) yields 231 decision flips at α=0.05, and every one favors promotion *relative to that reference*. The fallback is an uncorrected normal approximation; the one-line continuity correction replaces those 231 promotion-side flips with 9 of the opposite sign, on a disjoint set of cells, while mid-p leaves 221 anti-conservative against the same exact reference: the fix is a dial, not a restore. The largest absolute error is 0.152610 at b=14, c=13, a cell where the two-sided exact reference caps at 1.0 (partly definitional); away from the cap it is 0.150132 at b=14, c=12. In the discontinuity band the code returns $p = 0.049860$ where the reference returns 0.075519 (1.51×). A workspace-wide classifier finds 118 name-matched function entries, 82 of them paired-binomial tests; the defect is localised to two copies of one function, and a rejudgment of the system's fourteen-entry promotion ledger shows no recorded decision actually flips. A public-code survey of 68 `exact`-named implementations finds one that breaks the same contract above a sample-size threshold; the other 67 were not observed to do so. We report two defects in our own tools, delimit what the audit does not establish, and state the design space the audited rule sits in.

1 Introduction

Self-improving agent systems accumulate candidates — patched prompts, edited skill banks, re-written code — and must decide, for each one, whether it replaces the incumbent. That decision is made by an *acceptance rule*. Unlike a benchmark score, an acceptance rule is consulted by every later consumer of the system's history: whoever reads the promoted ledger inherits whatever the rule actually did, not whatever its name suggested.

This paper audits one acceptance rule from a production self-evolution harness in our workspace. The function is named `mcnemar_exact`; it gates promotion with

```
promote = bool(new_wins > old_wins and p < 0.05)
```

Its docstring reads (translated from the original Chinese) "two-sided exact p; exact below n=25, normal approximation otherwise." So the body is not hidden: a reader who opens the file learns the truth. But

the name is what the call site advertises, and the name claims a rigour the body does not honour. That is this paper's subject in miniature: **an acceptance rule that makes a rigour claim its body does not honour** — and, as we will show against the level-matched exact reference, the resulting error is not symmetric noise. It is a one-sided bias in favour of promotion.

The question has practical teeth, though we draw the bridge carefully. The *pattern* — promotion gates consulted once per candidate inside a self-improvement loop — is documented in the published literature: (He, 2026) analyzes 145 modifications eligible for analysis in the public Darwin Gödel Machine logs — six published runs, pooled — small-n territory exactly where an exact test and an uncorrected approximation disagree. That log is not this workspace and that rule is not this function; the citation establishes regime similarity, not shared history. And the same audit, run workspace-wide, shows the defect is not widespread *there*: it is two copies of one function, distinguished from 118 neighbouring implementations by precisely the thing the audit is built to surface — a disagreement between what a function is called and what it does. Public code supplies one further instance of the same defect (§4), which is what moves the finding from "two copies in one workspace" to "a class of error with at least two independent occurrences".

Our contributions:

1. **Measured one-sidedness, benchmark stated** (§3): recomputation of the entire fallback region against the level-matched two-sided exact reference — 231/55,469 decision flips, all favoring promotion relative to *that* reference. The qualification is load-bearing: against a one-sided α=0.05 exact reference the same region reads 928 conservative cells and none anti-conservative, so the direction claim is reference-relative and we say so.

2. **Mechanism and design space** (§3): the fallback is an uncorrected normal approximation; the continuity correction replaces the 231 promotion-side flips with 9 of the opposite sign, on a disjoint set of cells, while mid-p leaves 221 anti-conservative against the same reference, and the exact conditional test the audit treats as reference is itself known-conservative (Fagerland et al., 2013) — so switching to it trades bias for throughput rather than restoring truth.

3. **Pattern or accident** (§4): a static classifier over 118 name-matched function entries, finding the defect localised to two distinct copies of one function — the only three hybrids, and all three named `mcnemar_exact` — with the classifier's name-gated recall stated as a boundary, not buried.

4. **Dynamic validation and self-audit** (§5, §6): seven targets executed against the reference, including one whose largest error comes from a degenerate-tie artifact of its own correction term; two defects in our own tools, of the same class the audit hunts.

5. **Ledger rejudgment** (§10): the audited system's fourteen recorded gate decisions, re-evaluated read-only against the exact reference — all fourteen agree; the harm this paper measures is potential, and we show it.

6. **Our own instruments, described correctly** (§7): the audit CLI exits 0 on a target it has just declared non-exact, so we call it a *reporter* rather than a gate and scope the fail-closed claim to the selftest injection suite. We also report a correction to our own public-survey result: the first criterion asked whether an exact formula was present, and cleared 68 of 68; re-adjudicating for a sample-size switch finds one counterexample — an `exact`-named function in `Gurp-S/XEYO` that returns a normal approximation above n=500. The corrected rate is 1/68, reported as a floor and not as an ecosystem prevalence.

2 The gate under audit

The audited function lives at `evo_agent/frozen_v6/evolve.py:12`. Its promotion line (line 44 of the file) is

```
promote = bool(new_wins > old_wins and p < 0.05)
```

with p from `mcnemar_exact`. The McNemar test (McNemar, 1947) is the right instrument for paired binary outcomes — new-variant win/loss against the incumbent on the same tasks — so the choice of test is not the defect. The defect is the switch: below n=25 discordant pairs the function computes the exact conditional binomial tail; at $n \geq 26$ it silently switches to a normal approximation. Four statements of the same contract disagree:

Layer	Statement	Who sees it
Name	`mcnemar_exact`	every reader of the call site
Docstring	"exact below n=25"	readers who open the file
Boundary	docstring says n<25; body runs exact at n=25 too	readers who diff the two
Body	normal approximation at $n \geq 26$	nobody, unless they test

A note on orientation, since the counts carry sign: the function's docstring defines b as old-wins/new-losses and c as old-losses/new-wins, and its call site passes `(old_wins, new_wins)` — so the promotion side is the call site's second argument. The p-value is symmetric in (b, c); the scans below state the promotion side as $b \geq c$, and by mirror symmetry the two readings cover the same cells.

A name is the cheapest thing to trust and the most expensive thing to be wrong about, because everything downstream of the call site trusts it by default — a contract lens with a thirty-year genealogy (Meyer, 1992).

3 F-01: the approximation is one-sided — relative to its reference

`mcnemar_audit.py` recomputes every cell of the fallback region against an *independent* reference: `scipy.stats.binomtest` (Virtanen et al., 2020), the exact conditional binomial test (two-sided, default exact method). The region is the gate's own decision surface, scanned over $n = 26 \ldots 500$ with $b \geq c$ (the gate is directional; see the orientation note in §2):

Region n=26 … 500, $b \geq c$		62,819 cells
Cells the gate would promote		55,469
Decision flips at $\alpha = 0.05$	231	(0.42% of promotions at this bound)
Decision flips at $n \leq 200$	84	(1.02% of its 8,204 promotions)
All 231 anti-conservative (vs. two-sided exact)		**True**
Largest absolute error		0.152610 at b=14, c=13

The one-sidedness is the finding, stated at the level it holds. At the *decision* level, against the two-sided exact reference, there is no cell in the audited region where the approximation is stricter: all 231 flips point the same way, the code promotes where the exact test holds back. The direction is a deterministic consequence of the missing correction, not a random sign, and we report the count rather than a significance test. The 231 cells are an enumeration of one formula's boundary under a single reference, not independent trials, so a binomial tail on them has no null to condition on. A reviewer of a promotion ledger produced by this gate inherits a systematic bias, not a random one. Ties and near-ties are outside the count from the start: 475 cells in the region carry an exact two-sided p of exactly 1.0, so neither test can promote there and none of them contributes a flip. At the *p-value* level the claim would be false — in 41,076 of the 62,819 cells the approximation's p is *above* the reference's (all deep-tail cells where both tests reject; the largest code p among them is 5.3×10^{-5}, far from any threshold) — which is exactly why the paper's claims are stated in decisions, not numbers.

The direction claim is also reference-relative, and the reference must be named. The gate's promote condition applies one direction of a two-sided test, and its α=0.05 threshold doubles the one-sided tail — this is the *level-matched* comparator, the one whose rejection region the threshold was calibrated to, and against it the flips read anti-conservative. Against a one-sided α=0.05 exact reference the same region reads the other way: 928 conservative cells, zero anti-conservative (exemplar: at b=18, c=8, the code returns 0.049860 against a one-sided exact 0.037759). Which comparator is "right" depends on what the caller means by $p < 0.05$; what is not defensible is reporting a direction without naming the reference. We name it: two-sided, level-matched.

One qualification belongs to the comparison itself, and we state it rather than let a reader supply it. The exact conditional test is conservative (Fagerland et al., 2013), so at a common nominal α its achieved size is below 0.05; part of the 231-cell gap is therefore the reference's conservatism rather than the approximation's missing correction, and these data do not separate the two. The mid-p arm above is the nearest available check: it removes much of the exact test's conservatism and still leaves 221 of the 231 cells anti-conservative, so the direction is not an artefact of the reference's size alone. What a full separation would require is an actual-size-matched comparison — a randomized exact test, or an exact level calibrated to the approximation's achieved size — and that is a different measurement from the one this audit makes.

The mechanism is textbook, and naming it sharpens the finding. The audited fallback computes $z = |b-c|/\sqrt{b+c}$ and $p = 2\,Q(z)$ — an uncorrected normal approximation, whose missing $\pm\frac{1}{2}$ continuity term is the classical cause of exactly this signature: p pulled below the reference uniformly near the center, where the correction matters most. The correction is a one-line change, and the two standard repairs sit on opposite sides:

Variant	anti-cons. flips	cons. flips	max abs err	
audited (uncorrected normal)	231	0	0.152610	$(14, 13, 27)$
continuity-corrected χ^2	0	9	0.002053	$(19, 8, 27)$
exact, mid-p	221	0	0.149446	$(14, 13, 27)$
exact conditional (reference)	—	—	—	

The continuity correction replaces the promotion-side bias with a conservative one on a disjoint set of cells; mid-p fixes only ten of the 231 flips and leaves 221, each still anti-conservative against the same exact reference; and the exact conditional test itself is known to be conservative (Fagerland et al., 2013), so "switch to exact" trades anti-conservative bias for evolution throughput rather than restoring a true nominal level. The design space is a dial — uncorrected, corrected, mid-p, exact — and each position buys a different error profile. The direction is also robust to the threshold: sweeping α across $\{0.01, 0.025, 0.05, 0.10\}$ yields 215, 224, 231, 233 anti-conservative flips. And the corrected variant overshoots where the audited one erred: at the band's worst cell (b=18, c=8) it returns 0.077556 against the exact 0.075519. None of this machinery is new; the contribution is the audit trail that forces a gate to say which dial position it occupies.

Three worked examples from the discontinuity band. The largest understatement among the 231 flips — the sharpest, measured as the ratio of exact to code p at a flip cell — is 1.525 at b=19, c=8:

b	c	n	code p	exact p	exact/code
18	8	26	0.049860	0.075519	1.51×
19	8	27	0.034264	0.052239	1.52×
22	10	32	0.033895	0.050102	1.48×

At b=18, c=8 (n=26) the code returns $p = 0.049860$ — barely under the threshold, a promotion — where the exact test returns $p = 0.075519$, comfortably above it. The reference is not home-made: `scipy.stats.binomtest(8, 26, 0.5).pvalue` returns 0.07551869750022888, and the gate returns 0.049860.

The flips are a threshold-band phenomenon, not a diffuse smear: every one of the 231 has exact p in (0.050005, 0.075519], and 218 of them in (0.05, 0.06]. The code's error never reaches two cells' width in $b - c$ anywhere that matters — it only has to reach one threshold. The 0.42% headline is correspondingly a bound-dependent summary, not an invariant: at $n \leq 200$ the same function flips 1.02% of its promotions.

The band's width says *where* the gate errs; the loop says *why it matters*. The two error columns are not symmetric in consequence. A false rejection discards one candidate — an opportunity cost paid once. A false promotion writes the promoted variant into the ledger as an improvement, and the next generation starts from it: the variant is re-queried, forked, and serves as the comparison baseline, so the error is inherited by everything downstream. The compounding arithmetic is a scenario, not a measurement, and we label its two inputs. If each of 145 decisions independently carried the gate's nominal one-sided budget $q = 0.025$,

at least one false promotion would occur with probability $1 - (1 - 0.025)^{145} = 0.9746$; that is a property of any $\alpha = 0.05$ gate run 145 times. If instead each decision carried the defect's cell-level flip proportion $231/55{,}469 = 0.42\%$, the same arithmetic gives 0.454. Neither number is an estimate of this gate's field risk: the 145 are modifications in the published Darwin Gödel Machine logs, not this gate's decisions, and the 0.42% is a proportion over enumerated promotion cells, not a probability under the distribution of real candidates. What the enumeration does establish is that 0.42% of the cells this gate would promote are decided the other way by the exact reference; how that translates into ledger risk depends on a candidate distribution this paper does not have. A 0.42% decision-level flip rate is easy to dismiss as a rounding concern; the compounding arithmetic is why it is not. The pattern — flat accounting that hides where the cost actually lands — is the same one that makes per-token cost accounting matter in agent loops; acceptance gates deserve the same treatment.

The switch itself is worth pausing on. For $n \leq 25$ the function runs its exact branch, and that branch is transcription-verified against the reference; the exhaustive dynamic audit of §5 scans only $n \geq 26$, so the exact branch's correctness on its own domain rests on transcription and spot checks, not on the same cell-by-cell recomputation we demand of the approximation. The boundary is invisible at the call site — the caller passes a pair of win counts and receives a number that changes character halfway through its domain.

4 F-02: one file, or a pattern?

Is the defect one file or a family trait? `acceptance_gate_audit.py` walks the workspace tree, finds every function whose name looks like a paired binomial test, and classifies each body *statically* — exact tokens versus asymptotic tokens — before any dynamic check. The classifier is name-gated, and that is a boundary, not an implementation detail: a body that reaches an approximation through a helper it does not recognise, or hides one under an innocuous name, is invisible to it; conversely a correct exact test named `mcnemar_fast` would never be flagged. Its recall is a property of the workspace's naming conventions, the 48 unclassifiable helpers below are a standing blind spot, and the dynamic audits of §5 mitigate — they do not prove absence.

Over the whole workspace (inventory frozen at `data/f02_gate_inventory.json`; see §A for a re-run delta of +3 caused by our own artifact archive):

Class	Count	Contents
exact-only	60	every `mcnemar_scipy`, every `mcnemar_two_sided`
unknown	48	test helpers, power/n/OR calculators, `chi2` wrappers
approx-only	7	all seven are `mcnemar_chi2` (Yates-corrected chi-square)
hybrid	3	**all three are named `mcnemar_exact`**

The three hybrids (errors at the $n \leq 200$ dynamic bound):

Path	line	flips ($n \leq 200$)	max abs err	verdict
`evo_agent/frozen_v6/evolve.py`	12	84	0.1526	not exact
`evo_agent/evolve.py`	14	84	0.1526	not exact
`mcnemar_audit.py`	48	—	—	the transcription this audit uses

The honest headline is *not* "the workspace's gates are broken." Sixty implementations are exact-only, including every `mcnemar_scipy` and every `mcnemar_two_sided` in the `gsm8k-self-evolve` and `paper_workflow_kb` trees, and `paper_joint_mem_forget_formal`'s own `mcnemar_exact`. The defect is localised to two copies of one function in `evo_agent` — and the two files are genuinely distinct (`evo_agent/evolve.py`: 6,486 B, mtime 2026-09-13 — pre-repair metadata; the live copy was repaired on 2026-10-17, see §10; `evo_agent/frozen_v6/evolve.py`: 3,421 B, mtime 2026-09-10), not a symlink, so two copies exist; which was copied from which is not recorded, and we do not claim a direction. A name/body disagreement is exactly what the audit is built to surface, and it surfaced exactly three, two of which are real. (The third is the verbatim transcription the audit itself carries; the two `reproduces evolve.py` assertions in its selftest exist to catch that transcription drifting from its source.)

The 48 "unknown" rows are now resolved as well, and none hides an approximation. Twelve of them are (b, c)-callable and were audited dynamically at the $n \leq 200$ bound: five two-parameter copies (`mcnemar_two_sided` / `mcnemar_exact` across `ai-researcher` and both trees of `paper_joint_mem_forget_formal`) come back exact with zero decision flips and max abs error at most 9.3×10^{-14}, and seven `exact_mcnemar` copies across seven `stats.py` files — whose $(b, c,$ alternative) signature still audits cleanly because the third parameter defaults to `"two-sided"` — come back exact with zero flips and max abs error 1.4×10^{-13}. The remaining 36 are not p-value gates at all: 21 power/n/OR calculators whose inputs are effect sizes and target power, 12 test or selftest assertions, and 3 nested closures or recomputers that delegate to an exact implementation. The resolution also runs the classifier's reverse blind spot from the inside: the exact $2 \times$ min-tail pattern recurs across seven additional copies that the name-gated pass left unclassified, so "unknown" turned out to contain more exact code, not less.

Is this failure a pattern or an anecdote? A public-code presence survey (methods, per-source raw responses, and all fetched bodies are archived with the reproduction materials) searched Sourcegraph and grep.app for `mcnemar` and found 17,335 line matches across 5,746 files on Sourcegraph, of which 1,020 across 277 files name `mcnemar_exact` itself. Seventy-nine files were fetched, 78 of which yielded 81 non-test implementation definitions classified at function-body level; the seventy-ninth was fetched only as a delegate target and contributes no definition. The 78 are the files whose search hit exposed a body that could be fetched, listed one by one in `files-manifest-exact-definers.json` (which also records the seventy-ninth, a delegate target outside the 277-file population); they are a convenience sample of the 277 candidates rather than a random draw, so the count below is a within-sample count and not a prevalence estimate. The screening record and every fetched body are archived with the reproduction materials. Sixty-eight definitions carry *exact* in the name: 61 contain a direct exact implementation (combinatorial tail sums, `binomtest`, fraction or log-space arithmetic), four delegate honestly to exact helpers, two wrap exact routines, one formats without computing. That first criterion, applied to a body read alone, cleared all 68, because it asked only whether an exact formula was present. On that reading, the approximation bodies the survey did flag sat under names that make no exact promise (`mcnemar_normal`, `mcnemar_chi2_cc`, `mcnemar_asymptotic`), and one evaluation harness co-reports exact and asymptotic p-values under separately named functions in the same file. The reading was blind to a sample-size switch inside an otherwise exact body.

But one of the 68 does break its name, and our first criterion missed it. Re-adjudicating the same 68 definitions under a criterion that looks for a *sample-size switch* rather than a *formula* returns one counterexample. `Gurp-S/XEYO/python/evals/changedetect/stats.py` defines `mcnemar_exact(b, c)`, sets `_EXACT_MAX_N = 500`, and at `stats.py:67-68` returns `mcnemar_normal(b, c)` whenever $n = b + c > 500$ — the continuity-corrected normal approximation, i.e. the same defect class as the F-01 target, in a public repository, under a name that promises exactness. The original classification recorded this function as `exact` because its small-n body is exactly a `math.comb`-based tail sum. **The first criterion was wrong, not the code**: it asked "is there an exact formula here", which this file answers yes to, instead of "does the function return the exact tail for every input it accepts", which it does not.

That correction changes the headline from *absence* to a *counterexample count*, and it also sharpens the finding in a way the absence claim could not. The counterexample is the same defect as the one this paper audits — an `exact`-named gate that silently degrades past a threshold — discovered in independent public code by a criterion that only became visible once the F-01 target taught us what to look for. So we state the corrected result: **1 of 68 exact-named implementations in a 78-file public sample silently substitutes a normal approximation above a sample-size threshold; the remaining 67 were not observed to do so.** A second file, `NVIDIA/SkillEvaluator`, is the instructive contrast: it computes the tail in exact rational arithmetic (`Fraction`, `collector.py:4587`) with no switch at all, and handles float underflow by returning `None` plus a separate "minimum attainable p" instead of degrading the p-value — the same numeric pressure, handled by disclosure rather than substitution.

What the corrected count does and does not support. It is not a prevalence estimate. The source population (1,020 line matches across 277 files naming `mcnemar_exact`) is not a frame from which 78 files were drawn at random; no confidence interval is computed; two files were unreachable; and the 68 aggregate four categories of unequal evidentiary strength (61 body reads, 4 delegate-verified, 2 wrappers, 1 that computes no

p-value and therefore cannot fail the test it is counted in). That last category leaves an effective computational denominator of 67, although we keep "1 of 68" so the denominator remains tied to the name-selection criterion. A presence claim survives all of this — the defect exists in public code, independently of this workspace — and a rarity or rate claim does not. The counterexample also confirms the contract violation, not the direction result: its fallback is the continuity-corrected normal, which §3 shows runs conservative, so we do not claim the XEYO instance favors promotion. The qualitative reading we do stand behind is the gradient: the name is usually honest, which is exactly why a name that is not escapes notice.

A further axis this survey does not measure. The correction from 0 to 1 out of 68 is a count on the exactness axis only. Sidedness is not instrumented (§11, last row): a function correctly named for exactness can still hand a one-sided p-value to a caller expecting two-sided. And the $n \leq 200$ dynamic bound used for the *workspace* targets does not apply here at all — these 68 were read statically, and the counterexample above is precisely a case where a static read at small n is silent about the large-n branch. We report the count as observed within this convenience sample, not as a rate among `exact`-named functions at large.

5 Dynamic validation of the classifier's positive claims

A static classifier that says "exact-only" is itself making a claim, and a claim needs testing. Six exact-only implementations plus one asymptotic contrast were loaded and executed against the same reference (all rows at the $n \leq 200$ dynamic bound; the asymptotic row's flip count grows to 9 at $n \leq 500$):

Target	flips	max abs err	exact	note
`paper_workflow_kb`: `mcnemar_scipy`	0	0.0	yes	
`paper_promotion_gate`: `mcnemar_scipy`	0	0.0	yes	
`gsm8k-self-evolve`: `mcnemar_two_sided`	0	2.0×10^{-15}	yes	float noise
`joint_mem_forget_formal`: `mcnemar_exact`	0	2.0×10^{-15}	yes	float noise
`csa2_robust`: `mcnemar_binomtest`	0	0.0	yes	
`retrieval-demo` (w3): `mcnemar_exact`	0	5.0×10^{-5}	yes	`round(..., 4)`
`paper_workflow_kb`: `mcnemar_chi2`	5	0.1555	**no**	flagged asymptotic; see below

Targets are abbreviated; full paths are recorded in `FINDINGS-F01.md`, §4, and the frozen per-target outputs in `dynamic_audit_frozen200.json`.

The last row's error deserves its own paragraph, because it is not the asymptotic gap the "asymptotic" label predicts. The audited implementation computes $\chi^2 = (|b-c|-1)^2/(b+c)$ *without clamping the correction at ties*: when $b = c$ the term evaluates to $1/n$ instead of 0, and the function returns $p = 0.8445$ where the correct value is 1.0 — an error of 0.155481 at $(13, 13, 26)$, larger than the transcribed `mcnemar_exact`'s worst error (0.152610). Decomposing the $n \leq 200$ region: its 88 tie cells contribute that 0.155481 artifact, while away from ties the same continuity-corrected form errs by at most 0.002053 (at $19, 8, 27$). The tie defect never changes a decision — both p-values hold — but it is a second, independent instance of this paper's subject: a formula whose boundary case silently disagrees with its reference, in the very implementation that represents the "fixed" end of the dial in §3. Its five decision flips (nine at $n \leq 500$) are all conservative-direction.

One target could not be loaded at all: `retrieval-demo/bline/w2_harness.py` imports a sibling module (`w1_hybrid`) that is not on `sys.path`. The tool **refuses and exits** with a non-zero status (2, `FAIL-CLOSED`) rather than reporting a clean result. Note the scope of this guarantee: it covers *execution* failure — a target that cannot be imported or called, an empty inventory, a vacuous $n_{\max} < 26$ region. It does not cover *verdict* failure; see §7.

6 Two defects in our own tools

Both defects are the same class of error the audit hunts — a discrepancy between what a check claims and what it does — which is why they are reported as first-class results.

1. Display rounding conflated with approximation. `retrieval-demo/bline/w3_harness.py` computes the exact tail correctly and then rounds the output to 4 decimals, giving 5×10^{-5} of benign error. The first version of our exactness check used a 10^{-12} tolerance and would have reported that *correct* exact test as non-exact. The fix splits the question in two: `decision_flips` (the decision-relevant question: does the verdict change?) and `max_abs_error` (reported separately), with an explicit `EXACT_TOL = 1e-4` and an `output_rounded` flag. The tolerance is not arbitrary: 1×10^{-4} is twice the largest benign error a 4-decimal display can introduce (5×10^{-5}), and it sits more than three orders of magnitude below the errors the approximations actually produce: 0.149446 for mid-p, 0.152610 for the audited target, and 0.1555 for the continuity-corrected copy at $(13, 13, 26)$, of which at most 0.0021 is asymptotic gap and the rest is the tie artifact.

2. An injection that detected nothing, silently. The `acceptance_gate_audit.py` gate's `--inject no-flips` branch body was `pass`: the injection changed nothing, the gate exited 0 — meaning "not detected" — and printed nothing about it. A self-test that passes while its fault injection does nothing is worse than no self-test. It was replaced with `skip-renamed` (an auditor that walks past every function named `mcnemar_exact`), which the "scan locates the F-01 target" invariant does catch. Two further guards came out of the same pass: a `most_severe_flips[0]` index on an empty list that crashed two injections instead of failing them, and a band assertion corrected because the most severe flip sits at n=32, not at the switch itself.

A third injection is kept permanently in the suite: `same-reference`, where the reference *is* the audited function. An audit whose oracle is its target can never find anything, so the gate must fail if that configuration ever occurs.

Read as a set, the injections are a control system for the auditor itself, and it is worth naming the mapping explicitly:

Injection	Control it provides
`skip-renamed`	exemption-resistance: an auditor that walks past the F-01 target must not look clean
`same-reference`	circularity: an oracle equal to its target can never detect anything
`conservative-bias`	direction-sensitivity: the pipeline must react to flips in *either* direction
`empty-scan`	degenerate input must fail closed, not report a clean bill

One retired injection taught its own lesson: in `acceptance_gate_audit.py`, the original `no-flips` branch body was `pass` — a named control that controlled nothing — and it survived until a human read the outputs (defect 2 above). (`mcnemar_audit.py` still carries a working `no-flips` injection, which clears the flip list and is caught by the expected-flip-count assertion.) A control that cannot fail is decoration.

7 What the exit code means

One property of our own instruments needs stating before §8, because it is the exact defect this paper is about applied to the auditor itself. Read the two CLI branches of `acceptance_gate_audit.py` (lines 304–316 as pinned by the `sha256` prefix in §8):

- `-audit` calls `dynamic_audit`, returns 2 if the result is `None`, otherwise prints the JSON — **including** `"is_exact": false` and `"decision_flips": 84` — and **returns** 0 **unconditionally**.
- `-selftest -inject` is the branch that behaves as a gate: it returns 1 when an injected violation is detected and 0 when the injection is *not* detected, printing `*** NOT DETECTED -- GATE IS DECORATIVE ***`.

So the tool has two distinct exit vocabularies, and they are not the same vocabulary. On the `-audit` path, exit 0 means *the audit ran*, not *the target passed*. Re-measured on 2026-10-25: auditing `../evo_agent/frozen_v6/evolve.py:mcnemar_exact` returns exit 0 with `is_exact: false`, 84 decision flips over 10,019 cells and `max_abs_error: 0.15261`; auditing `../paper_workflow_kb/scripts/stats.py:mcnemar_chi2` returns exit 0 with 5 flips and `max_abs_error: 0.155481`. Both are *failures of the audited artefact* reported at exit 0.

We therefore describe `-audit` and `-workspace-scan` as **reporters**, not admission gates: they fail closed on execution errors (unimportable target, empty inventory, vacuous region — all verified at exit 2), and they record verdicts in JSON for a consumer to read. The phrase "fail-closed gate" in §8 applies to the selftest injection suite only. The `sha256`-pinned behaviour is unchanged from what produced the results in this paper; we are correcting the description, not the record.

Why we did not simply fix it. Making `-audit` exit non-zero on `is_exact: false` would be a three-line change, and it is tempting because it would let the tool's name keep its promise. We declined for two reasons. First, `is_exact: false` is the *expected* result for every target this audit was built to find — the F-01 target itself — so a non-zero exit on discovery would make the reporter unusable inside the very pipeline that consumes it; discovery and admission are different contracts and conflating them is how a gate acquires a name it cannot honour. Second, changing the exit semantics now would silently reinterpret the 2026-10-17 run recorded in App. A as having "passed" under a stricter contract, which it did not. The honest repair is a separate, explicitly-named admission mode with its own negative and positive tests; that is future work, and it is listed as such in §10.

8 Gate status

Both audit tools are part of the program's shared fail-closed gate suite (37 checks, all green as of 2026-10-17; tool state pinned by `sha256` prefixes `1227899ea8e2cf8a` for `mcnemar_audit.py` and `ac317080ea41af8d` for `acceptance_gate_audit.py`; every gate also runs fault injections that must trip it). The table separates the two exit vocabularies of §7:

Tool / mode	Status
`mcnemar_audit.py -selftest`	gate: clean exit 0 (14 checks); `no-flips`, `conservative-bias`, `same-reference` → exit 1
`acceptance_gate_audit.py -selftest`	clean exit 0; `empty-scan`, `no-hybrids`, `mislabelled`, `skip-renamed`, `dynamic-clean`, `vacuous-exact` → exit 1
`mcnemar_audit.py -scan`	*reporter*: exit 0 after printing the scan JSON; no verdict-based non-zero exit
`acceptance_gate_audit.py -audit`	*reporter*: exit 2 on execution failure only (unimportable target, empty inventory, vacuous region); exit 0 otherwise, verdict in JSON
`acceptance_gate_audit.py -workspace-scan`	*reporter*: exit 2 on empty inventory only; exit 0 otherwise, inventory in stdout/JSON

The rows above the rule are *gates*: their exit codes report whether an injected violation was detected. The rows below it are *reporters*: they fail closed on execution error, then return 0 once the report is produced, whatever verdict it contains. `-audit` imports the target, prints `is_exact`, and returns 0 whether or not the target is exact; `-scan` prints and returns 0 unconditionally. A caller that needs a blocking gate must test `is_exact` itself or run the selftest injections; reading a reporter's exit code as a pass would repeat the name/behaviour gap this paper audits.

9 Related work

None of the statistics here is new, and the paper's debts are specific. McNemar's test became the standard instrument for paired classifier comparison through (Dietterich, 1998). The exact conditional test this audit

treats as its reference is itself known to be conservative, with mid-p and corrected asymptotic variants recommended in its place (Fagerland et al., 2013) — which is why §3 presents a dial rather than a repair. The wider discipline of testing when comparing learned systems — multiplicity, test-set reuse, significance inflation — is surveyed in (Dror et al., 2018); at the systems layer, flaky and retried tests in CI pipelines present the same accept-on-noise hazard (Luo et al., 2014). Acceptance gates for learning agents are the subject of the safe-reinforcement-learning literature (García & Fernández, 2015). The optional-stopping hazard of repeated testing is classical (Wald, 1945), and the racing/early-stopping acceptance machinery of algorithm configuration and HPO — F-Race/irace (López-Ibáñez et al., 2016), Hyperband (Li et al., 2017), ASHA (Jamieson & Talwalkar, 2016) — makes precisely the trade this paper audits: promote early on partial evidence. The contract lens on names is Meyer's design-by-contract (Meyer, 1992), imported here from module interfaces to statistical claims. At the agent layer, skill-library self-improvement (Wang et al., 2023), self-reflection loops (Shinn et al., 2023), and co-evolution traps (Feng et al., 2026) are the systems that would consume such gates; catalogs of self-modification decisions (He, 2026) are the loops that would generate their candidates; and LLM-as-judge reliability (Zheng et al., 2023) bounds the label validity any such audit must assume. The contribution is none of these instruments; it is the audit trail that binds one named gate to its measured behaviour — and the demonstration that both a one-line repair and a boundary case can hide behind a correct-sounding name.

10 What this does not establish

- **The audited system's promotion history has been rejudged — read-only, and the harm remains potential.** The fourteen entries of the audited system's evolution ledger (`evo_agent/EVOLUTION_LEDGER.md`, excluding the smoke-record block that the ledger itself marks as not counted toward gate statistics) were re-evaluated against the exact reference: all fourteen recorded verdicts agree (five promotions, nine holds; every entry sits at $n \leq 12$, inside the exact branch), and the recorded p-values match the reference to their display rounding (0.0078 vs. 0.0078125, 0.0156 vs. 0.015625). This was an analysis we ran, not a repair we made: whether to re-derive past promotions is the system owner's call, and repairing the function would still invalidate the ledger's provenance. *Repair record (owner-authorized, recorded after the audited snapshot):* after this draft entered internal review, the owner elected to repair the *live* gate `evo_agent/evolve.py` (exact branch extended to all n; sha256 `32ab7425...` → `bb2d3d2b...`); the frozen snapshot `frozen_v6/evolve.py` is deliberately left unrepaired — it is the audited object of this paper and the reproduction anchor. The post-repair dynamic audit of the live gate reads 0 decision flips, max abs error 2.0×10^{-15} at $n \leq 200$ and 2.7×10^{-15} at $n \leq 500$ (float noise), and the ledger rejudgment above is unchanged (14/14). Every number in this paper refers to the archived pre-repair sources (App. A). The loop-level exposure is also not zero in general: a gate consulted once per candidate accumulates threshold-adjacent risk, and at one-sided α=0.025, 145 independent decisions sitting exactly at the band edge would carry a $1 - (1 - 0.025)^{145} \approx 97.5\%$ chance of at least one false promotion — an illustrative worst case, not a property of this ledger, whose entries sit far from the band (at the defect's measured flip rate the same arithmetic gives $\approx 45\%$, §3).

- **Label validity is upstream of this paper.** The wins that feed b and c are judged by regex matchers (`judge_strict`, `judge_content` in the audited file); if a judge mislabels a task outcome, the counts are wrong before any test runs. The audit takes the recorded labels as given and does not measure the judges' agreement with ground truth (Zheng et al., 2023).

- **The static classifier reads names and bodies.** As stated in §4: recall is name-gated, helpers are a blind spot, and the dynamic audits are a mitigation, not a proof of absence.

- **The audit pins the target by value, and the value pins protect transcription only.** `mcnemar_audit.py` transcribes the audited function; its two `reproduces evolve.py` selftest assertions compare that transcription against *hardcoded* reference pairs ($p(18, 8) = 0.049860$, $p(19, 8) = 0.034264$, tolerance 5×10^{-7}; `mcnemar_audit.py:200-203`) — they never read the audited file, so they would not notice `evo_agent` drifting. The live guard is the dynamic audit of §5, which imports

the actual file; the honest statement is that between transcription checks and ad-hoc dynamic runs there is no standing watch.

- $n \leq 500$ **is a scan bound, not a natural limit.** The flip count grows with the bound: 84 flips at $n \leq 200$, 231 at $n \leq 500$ — and the flip *rate* moves too (1.02% to 0.42% of promotions), so single-number headlines are bound-relative in two ways.

- **One workspace, one defect body.** All counts are from a single research workspace's trees; the classifier's recall elsewhere is unmeasured, and a finding this localised (two copies of one function) licenses no prevalence claim about other codebases.

- **Neither tool is an admission gate, and no admission gate is built here.** As shown in §7, `-audit` and `-workspace-scan` exit 0 on a target they have just declared non-exact; only the selftest injection suite exits non-zero on a violation. Nothing in this paper blocks a promotion automatically — the two self-audit defects of §6 were caught by a human reading outputs, not by a gate rejecting a build, and the `no-flips` injection survived precisely because exit 0 was indistinguishable from success. A genuine admission mode would need a named contract, a non-zero exit on `is_exact: false`, and its own negative test (a target that must be rejected) and positive test (a target that must pass); we have designed none of those, so the claim is not made.

11 Conclusion

The intended reader is the author of the next acceptance rule, and the reviewer of the next self-evolution paper. The audit's recipe is four moves, each executable in this workspace:

Name / artifact	Executable property it claims	Fail-closed gate	Drift control
`..._exact`	flips = 0, max abs err $\leq 10^{-4}$ over the decision region	`-audit` imports and executes and prints `is_exact`; the non-zero exit is enforced by the selftest injection `dynamic-clean`, not by `-audit` itself	dynamic re-run against the live file
transcription	reproduces the audited source	selftest value pins (catch hand-edits only)	dynamic audit reads the file
classifier claims	static tokens + dynamic confirmation	`-workspace-scan` (reporter) + `-selftest` injections (gate)	frozen inventory with delta note
sidedness / α axis	—	*not yet instrumented*: the name determines more contract axes than this audit tests	—

A name suffix is a claim; a claim becomes an executable property (region recomputation against an independent reference); the property is **reported** by a tool that fails closed on execution error and records its verdict where a reader can see it; and drift is controlled by importing the live file rather than trusting a transcription. We stop short of calling the last step a gate, for the reason given in §7: a check that a human must read is a leverage point, not an admission rule. What the recipe found here: a gate whose name promised exactness, whose body delivered an uncorrected approximation, and whose every decision error — relative to the level-matched reference — favored the action the gate exists to permit; plus a continuity correction in the "fixed" variant that silently errs at ties, and a display-tolerance trap in our own first instrument. What the recipe cannot deliver is recall beyond recognisable names, prevalence beyond one workspace, or the repair decision, which belongs to the system's owner.

A Reproduction and data availability

All commands run from `agent-verifiability/`, the program root inside the workspace; targets like `../paper_workflow_kb/...` resolve to sibling project trees of the same workspace. Every command below was re-run for this manuscript; the gate commands exit 0 clean and exit 1 under every listed injection.

```
# F-01: one-sidedness of the approximation (231/55,469, all anti-conservative)
python3 wave2-F-acceptance/mcnemar_audit.py --selftest     # exit 0
python3 wave2-F-acceptance/mcnemar_audit.py --scan         # exit 0

# F-01 fault injections: each must exit non-zero (fail-closed)
python3 wave2-F-acceptance/mcnemar_audit.py --selftest --inject no-flips
python3 wave2-F-acceptance/mcnemar_audit.py --selftest --inject conservative-bias
python3 wave2-F-acceptance/mcnemar_audit.py --selftest --inject same-reference

# F-02: workspace-wide classifier (118 implementations; 3 hybrids)
python3 wave2-F-acceptance/acceptance_gate_audit.py --selftest
python3 wave2-F-acceptance/acceptance_gate_audit.py --workspace-scan

# Dynamic audit of the transcribed target (imports the live file)
python3 wave2-F-acceptance/acceptance_gate_audit.py \
    --audit ../evo_agent/frozen_v6/evolve.py:mcnemar_exact --n-max 200

# Dynamic audit of the chi2 variant (5 flips, tie-degeneracy max err 0.155481)
python3 wave2-F-acceptance/acceptance_gate_audit.py \
    --audit ../paper_workflow_kb/scripts/stats.py:mcnemar_chi2 --n-max 200

# Every number in Sec. 3's design-space table, the ledger rejudgment,
# the threshold-band split, and the software versions (revision_stats.json)
python3 wave2-F-acceptance/revision_stats.py

# Shared fail-closed gate suite (37 checks + fault injections)
bash shared/run_gates.sh
```

All commands above were re-run for this manuscript on 2026-10-17 with the exit codes shown. One honest delta: the live `-workspace-scan` today finds 121 implementations, not 118 — the three additional hits are all archival copies of code the program itself produced (the auditor inside the workspace deposit, the repaired live gate in the repair artifact, and the auditor's archived transcription); all 118 implementations of the frozen inventory are unchanged, and none of the headlines move. A second delta is a repair to the auditor itself: an `-n-max` below the region floor of 26 used to produce a zero-cell region that reported `is_exact=true` on no evidence; it now fails closed (injection `vacuous-exact` added to the selftest), and the auditor's `sha256` prefix above reflects the post-repair tool.

Data availability. Every number in this paper is checkable by recomputation from the printed formulas and the commands of App. A; no external archive is cited, so no archive is required to verify any claim. The canonical records live in the workspace (a deposit is in preparation and the DOI will replace these workspace paths at camera-ready): the findings record `wave2-F-acceptance/FINDINGS-F01.md`; the frozen audit outputs `data/f01_mcnemar_scan.json` (region scan), `data/f02_gate_inventory.json` (118-implementation inventory), `wave2-F-acceptance/ dynamic_audit_frozen200.json` (per-target dynamic outputs, $n \leq 200$), `wave2-F-acceptance/revision_stats.json` (design space, ledger rejudgment, band split, versions), and `wave2-F-acceptance/flip_band.json` (the 231 flips' exact-p distribution). The pre-repair sources of the audited gate are archived with SHA-256 manifest and the owner-authorized repair record under `wave2-F-acceptance/ prerepair-snapshot/` (`REPAIR-RECORD.md`; live gate before `32ab7425...`, after `bb2d3d2b...`; frozen snapshot `43802d68...` unchanged). Software pinned for recomputation: Python 3.12.3, SciPy 1.18.1, Ubuntu 24.04 x86_64; the reference is `scipy.stats.binomtest(k, n,`

`0.5, alternative="two-sided")` with the default exact (small-p) method. Seventeen-digit p-values are version-sensitive; the audit's own recomputation path (`revision_stats.py`) regenerates them end-to-end under the pinned versions.

Reference values used: `scipy.stats.binomtest(8, 26, 0.5).pvalue` = 0.07551869750022888 against the gate's 0.049860 at b=18, c=8; largest absolute error 0.152610 at b=14, c=13. The transcription's region scan and the dynamic import of the live file agree: 84 decision flips at $n \leq 200$, 231 at $n \leq 500$.

References

Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, pp. 1383–1392, 2018.

Morten W. Fagerland, Stian Lydersen, and Petter Laake. The McNemar test for binary matched-pairs data: mid-p and asymptotic are better than exact conditional. *BMC Medical Research Methodology*, 13:91, 2013.

Yukun Feng, Xingyao Wang, Jinghan Yang, Xiangru Shi, Pengfei Xie, Philip S. Yu, Xian Wu, and Jiayi Ye. Evolving together, failing apart: The skill–policy co-evolution trap in self-improving agents. *arXiv preprint arXiv:2610.20845*, 2026.

Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(1):1437–1480, 2015.

Ruiqi He. Does failure feedback direct agent self-modification? *arXiv preprint arXiv:2610.14662*, 2026.

Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 240–248, 2016.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18 (185):1–52, 2017.

Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Thomas Stützle, and Mauro Birattari. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.

Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pp. 643–653, 2014. doi: 10.1145/2635868.2635920.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Bertrand Meyer. Applying "design by contract". *IEEE Computer*, 25(10):40–51, 1992.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2): 117–186, 1945.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Li, Di Zhu, Hao Zhang, Sewon Zhuang, Yue Xie, Yubo Wu, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.